

Abstract Interpretation

Is my program correct ?

- Hoare Logic (axiomatic semantic)
 - Deductive reasoning
 - SMT Solving
- Abstract interpretation

Semantic importance example

C_program:

```
int i, a;  
a = 5;  
for (i = 0; i < a; i++) {  
    a--;  
    printf("i = %d", i)  
}
```

will return i = 0, i = 1, i = 2

Ada:

```
A: integer;  
begin  
    a:=5;  
    for I in 0..A loop  
        A:= A - 1;  
        Pat("i=");Pat(I);  
    end loop;  
end;
```

will return i = 0, i = 1, i = 2, i = 3, i = 4, i = 5 (because it works with a copy of A)

In this chapter, we will use some kind of automata to define the correctness of our program ??

Semantics:

- use automata

(give here an example of automata for the C_program and the Ada program given according to copy of A in for loop)

Example

Java:

```
{  
    int a;  
    for (int i = 0; i <= a; , i++){  
        // blablabla  
    }  
}
```

Compiler

- refuses to compile the file
- emits a strong warning: "Variable 'a' might not have been initialized"
- So is not sure !

⇒ Problem of detecting non-initialized variable is not decidable (not computable) !

The Java compiler gives an approximation

- this is an 'over' approximation

(graph here with [programs: non initialized variable] in a circle, inside a circle with [programs rejected by the compiler] !) The set of rejected programs is bigger than the one with accurate answers ! It's a conservative approximation

- if the program is not rejected, we are sure there is no problem on it
- otherwise we don't know

Abstract $\approx$ Abstraction

Collecting Semantics (Hoare 69, Floyd 67)

On this chapter, we will focus on a small language that is the same as before except function calls. Contains:

- integers/boolean/variables/expressions
- if ..., while...

We have seen the operational semantic (quick recall here on operational semantic, especially on Mem and Val)

The compiler compile the program and build a data structure which is called the Abstract Syntax Tree (AST) which represents the program In fact, we can implement Mem and Val directly on the AST, which is called Interpreter Execution: execution of the program step by step

Here we will use other data-structure to represent the program, like automaton which is called the Control Flow Graph (CFG) of the program.

CFG: we take a program and we create interpreted automation of the program (S, T, s_0, V)

- S : set of states (finite)
- T : transitions $T \subseteq S \times (A(V) \cup C(V)) \times S$
- s_0 : initial state $s_0 \in S$
- V : set of variables (finite) of the program
- $A(V)$: assignments
 - $x := \text{Exp}$ with $x \in V$, Exp on V
- $C(V)$: conditions
 - boolean expression
 - build on V

Give an example here !

```
a = 5;
while i <= a do
  i = i + 1;
  a = a - 1;
done
```

Build automata graph of the given program here !

Collecting semantics

We will take a program, build the CFG and then express the semantics on the CFG

State of the CFG is usually called the Control Point (CP)

We will define (Try to clarify here the definition !!!): $\alpha : CP, V(\alpha) \rightarrow \forall x \in \text{Variable}, V(\alpha)(x)$ is the set of every possible value of reached while executing the program at α

Example with the automata that you must build just before the ==...:

- $V(A)(i) = V(A)(a) = \text{every possible value } \mathbb{Z}$

- $V(B)(i) = \{0\}, V(B)(a) = \text{every possible value } \mathbb{Z}$
- $V(C)(i) = \{0, 1, 2, 3\}$
- $V(C)(a) = \{5, 4, 3, 2\}$
- $V(F)(i) = \{3\}$
- $V(F)(a) = \{2\}$
- $V(D)(i) = \{0, 1, 2\}$
- $V(D)(a) = \{5, 4, 3\}$
- $V(E)(i) = \{0, 1, 2\}$
- $V(E)(a) = \{4, 3, 2\}$

Computation of V

If I have a transition with an assignment, I just look at what's happen on x :

- $V(B)(x) = V(A)[x := \text{expr}]$
- $V(B)(y) = V(A)(y), y \neq x$

If I have a transition with a boolean expression, we compute intersection with previous

- $V(B) = V(A) \cup [x > 10]$
 - $V(B)(x) \leftarrow \text{update}$
 - $V(B)(y) = V(A)(y)$ Particular case if condition is respected
- $V(B)$ may have several changes

If we merge state A and B in state C , we will have:

- $X_A = V(A)[x := y + 1]$ First we compute the effect of the assignment A
- $X_B = V(B) \cap [x < y]$ We do the same for the second transition
- $V(C) = X_A \cup X_B$ Then we compute the union of the two

Now, we come back to the example We obtain that

- $V(A) = T(ToP)$
- $V(B) = V(A)[i := 0]$
- $V(C) = V(B)[a := 5] \cup V(E)[i := i + 1]$ Here it introduces a loop because we need to compute $V(E)$ that needs $V(D)$ that needs $V(C)$ that we try to compute !!! (Make a small graph here that show the loop, please)
- $V(D) = V(C) \cap [i \leq a]$
- $V(E) = V(D)[a := a - 1]$
- $V(F) = V(C) \cap [i > a]$

We obtain a system of equations

Fixed-point equation system

Equation where we try to solve it when it depends on itself $V = \varphi(V)$ We are interested to know

- fixed-point defined ?
- can we compute it ? If not, what's about abstraction
- does this Fixed-Point (FP) equation always have a solution ?

Fixed-Point Theory (some ???)

We want to obtain a solution of $X = \varphi(X)$

Lattices

Poset: it's a partially ordered set

Lattice: it's a poset such that the infimum and supremum on 2 elements are total functions in E , the poset.

$$\inf : E \times E \rightarrow E \quad \sup : E \times E \rightarrow E$$

For $A, B \in E$, $\sup(A, B)$ is the **least upper bound** of A and B

- $\sup(A, B)$ is an upper bound (of A and B)
 - $A \leq \sup(A, B)$
 - $B \leq \sup(A, B)$
- $\sup(A, B)$ is least one
 - Let X be an upper bound of A and B . ($A \leq X$ and $B \leq X$)
 - $\sup(A, B) \leq X$

For $A, B \in E$, $\inf(A, B)$ is the **least lower bound** of A and B etc...

For instance, for a poset $(E, \leq)$, even if A and B are not comparable, we can compute $\inf(A, B)$ and $\sup(A, B)$.

A complete Lattice $(E, \leq)$ is a lattice such that $\forall y \subseteq E$, $\sup y, \inf y$ exist in E

If E is finite and E is a lattice then E is a complete lattice. When we have a finite number of elements, we can pick a subset of E that is finite. So:

- $(y = \{y_1, y_2, \dots, y_N\})$
- $\sup y = \sup\{y_1, y_2, \dots, y_N\} = \sup\{y_1, \sup\{y_2, \dots, y_N\}\}$

If $(E, \leq)$ is a lattice but not a complete lattice, then it is infinite. (Example in exercises)

$V(\alpha)(x) \subseteq \mathbb{Z}$ Parts of $\mathbb{Z}$ $\mathcal{P}(\mathbb{Z})$. Example: $(\mathcal{P}(\mathbb{Z}), \subseteq)$ is a complete lattice. $A, B \subseteq \mathbb{Z}$.

- $\inf(A, B) = A \cap B$
- $\sup(A, B) = A \cup B$

Exercise: prove that $\inf$ is $\cap$ and $\sup$ is $\cup$.

Now, let show that $(\mathcal{P}(\mathbb{Z}), \subseteq)$ is complete.

Let $y \subseteq \mathbb{Z}$. We have to prove that $\inf y, \sup y$ are defined with values in $\mathcal{P}(\mathbb{Z})$.

$$B = \{x \in \mathbb{Z}, \exists S \in Y, x \in S\}$$

- B is an upper bound of Y Let $S \in Y$ and $x \in S$. Then $x \in B$. So $S \subseteq B$. So B is an upper bound of Y .
- B is the least one. Let X be an upper bound of Y . Then $\forall S \in Y, S \subseteq X$. Let $x \in B$. By definition, $\forall S \in Y, x \in S \subseteq X$. So $x \in X$. So $B \subseteq X$.

$$B = \sup Y = \bigcup_{S \in Y} S$$

- The complete lattice of signs (intervals) $\mathcal{S} = \left\{ \leq 0, \geq 0, < 0, > 0, \neq 0, = 0, \underbrace{\top}_{\sup \text{ top}}, \underbrace{\perp}_{\inf} \right\}$

$V(\alpha)(x)$. We will use signs to abstract away the values of x .

The order on $\mathcal{S}$: left is less precise than right elements.

(Here draw a cube with all relations between elements of the set $\mathcal{S}$ with $\perp$ to $\top$ with sign combinations...)

Monotonicity: non-decreasing functions

$f : (E, \leq) \rightarrow (F, \leq)$ posets.

f is monotone/non-decreasing iff:

$$\forall x, y \in E, x \leq y \implies f(x) \leq f(y)$$

Theorem Knaster-Tarski (1928, $(\mathcal{P}(E, \subseteq))$, 1959: general)

First assumption

Let $(E, \leq)$ be a complete lattice (CL).

$f : E \mapsto E$ a monotonic function

1. The set of fixed-points of f , $FP = \{x \in E, x = f(x)\}$
2. In particular f admits a least fixed-point $\inf FP \in FP$ and a greatest one $\sup FP \in FP$.

Scott-Continuity

Need for next step

- $f : (E, \leq) \longrightarrow (F, \leq)$ posets.
- $(E, \leq)$ and $(F, \leq)$ are complete lattices.
- f is Scott-continuous iff for every non-decreasing sequence, $(u_n)_{n \geq 0}, \forall n, u_n \in E \implies u_n \leq u_{n+1}$
We have $f(\sup_{n \geq 0} \{u_n\}) = \sup_{n \geq 0} \{f(u_n)\}$

Counter-part of Knaster-Tarski theorem: Kleene theorem

Allow to get values

- $(E, \leq)$ a complete lattice (CL).
- $f : E \mapsto E$, f is Scott-continuous.
- The least fixed-point of f is equal to $\sup_{n \geq 0} \{f^{n(\perp)}\}$ where $\perp = \inf E$
- The greatest fixed-point of f is equal to $\inf_{n \geq 0} \{f^{n(\top)}\}$ where $\top = \sup E$